

MERN STACK INTERNSHIP – DAY 5 ASSIGNMENT REPORT

Company

Insta Dot Analytics

Submitted By

Name: Chayan Soni

Roll No: PU02325EUGBTCS086

Course: B.Tech (CSIT), First Year

College: Symbiosis University of Applied Sciences

Submission Date: 05-06-2026

FOR SOURCE CODE:

GIT HUB :- <https://github.com/chayan-soni/MERN-Internship>

Assignment Overview

Core Focus Areas

1. Production Front-End Optimization

Preparing React applications for deployment by generating optimized production builds, managing environment variables, and optimizing static assets.

2. Robust Express API Hardening

Implementing centralized error-handling middleware to manage runtime exceptions, provide structured error responses, and maintain server-side error logs.

3. Persistent State Management

Understanding the importance of maintaining application state across multiple routes and components in modern frontend applications.

Task 5.1 – Production Front-End Optimization

Objective

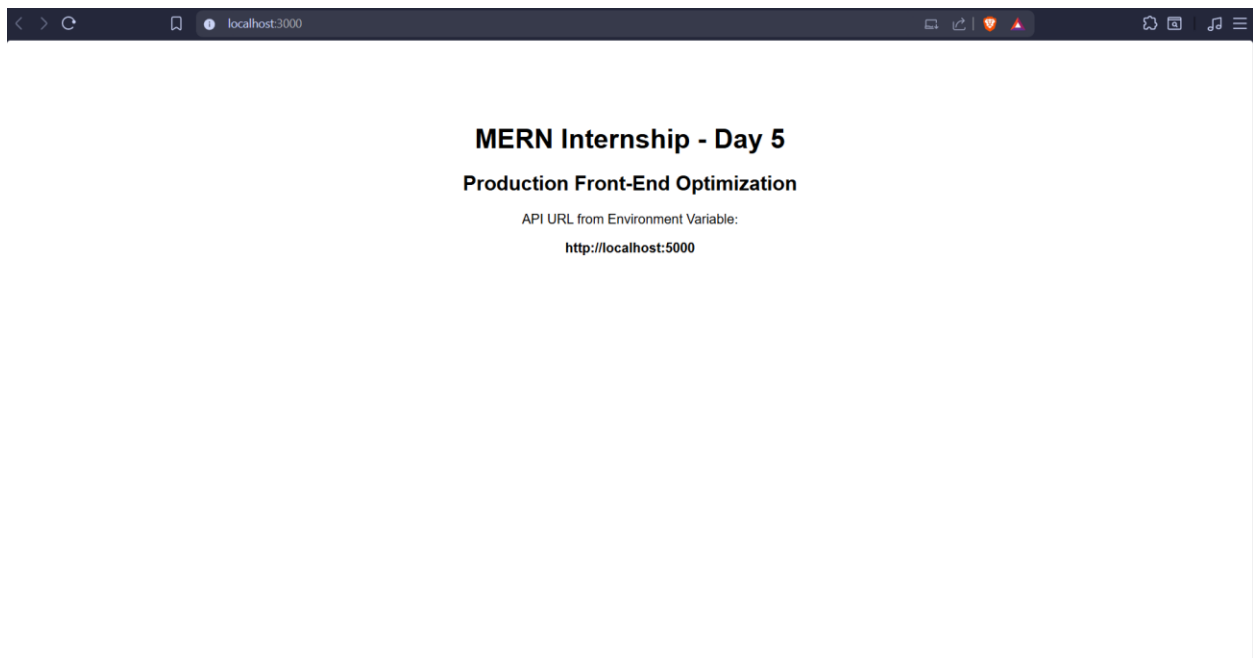
To prepare a React application for production deployment by generating an optimized build and configuring environment variables.

Implementation Steps

Step 1: React Application Setup

A React application was created to simulate a simple dashboard interface displaying environment variable data.

Screenshot 1: React Application Running



Step 2: Environment Variable Configuration

A .env file was created containing:

```
REACT_APP_API_URL=http://localhost:5000
```

The variable was accessed within the React application using:

```
process.env.REACT_APP_API_URL
```

Step 3: Production Build Generation

The following command was executed:

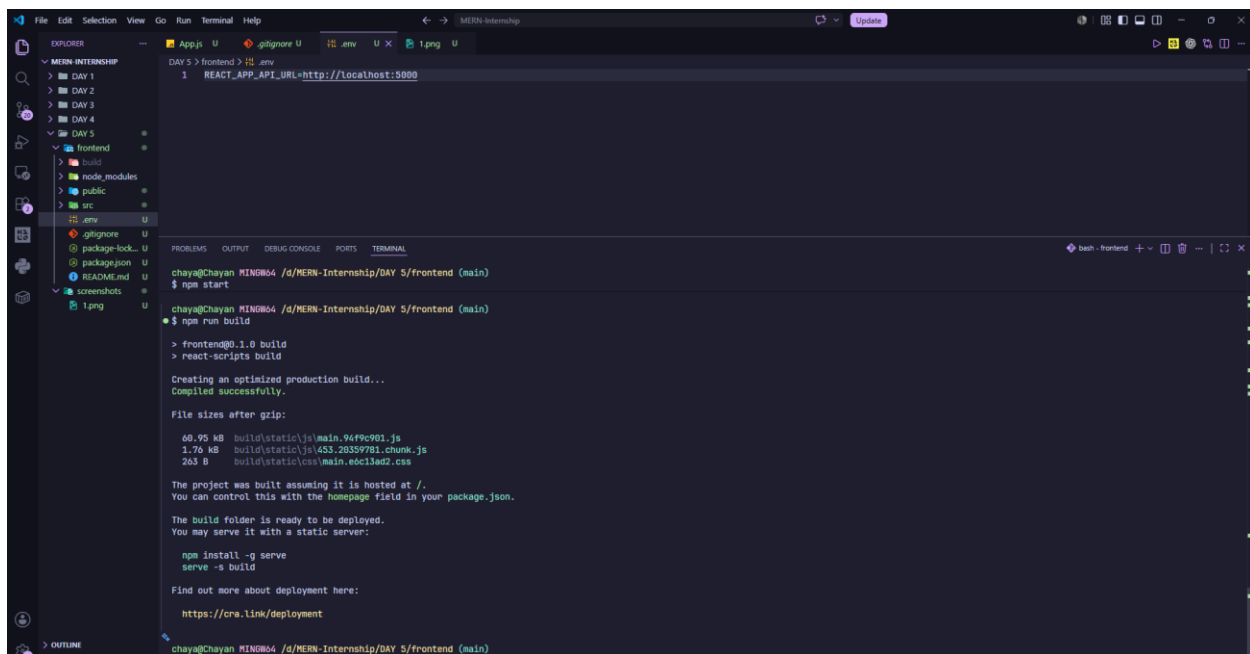
```
npm run build
```

This generated an optimized production build inside the build directory.

Benefits:

- Minified JavaScript files
- Optimized CSS assets
- Reduced application size
- Faster loading performance

Screenshot 3: Production Build Terminal Output



```
chaya@Chayan MINGW64 /d/MERN-Internship/DAY 5/frontend (main)
$ npm start

chaya@Chayan MINGW64 /d/MERN-Internship/DAY 5/frontend (main)
$ npm run build

> frontend@0.1.0 build
> react-scripts build

Creating an optimized production build...
Compiled successfully.

File sizes after gzip:

  60.95 kB  build/static/js/main.94f9c901.js
  1.76 kB   build/static/js/453.28359781.chunk.js
  263 B     build/static/css/main.eec13a02.css

The project was built assuming it is hosted at /.
You can control this with the homepage field in your package.json.

The build folder is ready to be deployed.
You may serve it with a static server:

  npm install -g serve
  serve -s build

Find out more about deployment here:
  https://cra.link/deployment

chaya@Chayan MINGW64 /d/MERN-Internship/DAY 5/frontend (main)
```

Learning Outcomes

- Learned how React production builds are generated.
- Understood the role of environment variables.
- Learned how build optimization improves deployment readiness.
- Understood the purpose of static asset optimization.

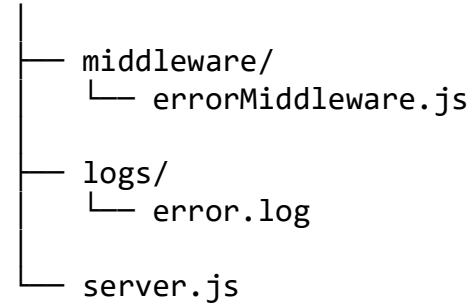
Task 5.2 – Centralized API Middleware

Objective

To implement a centralized error-handling middleware system in Express.js that catches errors, generates structured responses, and records error logs.

Project Structure

backend/



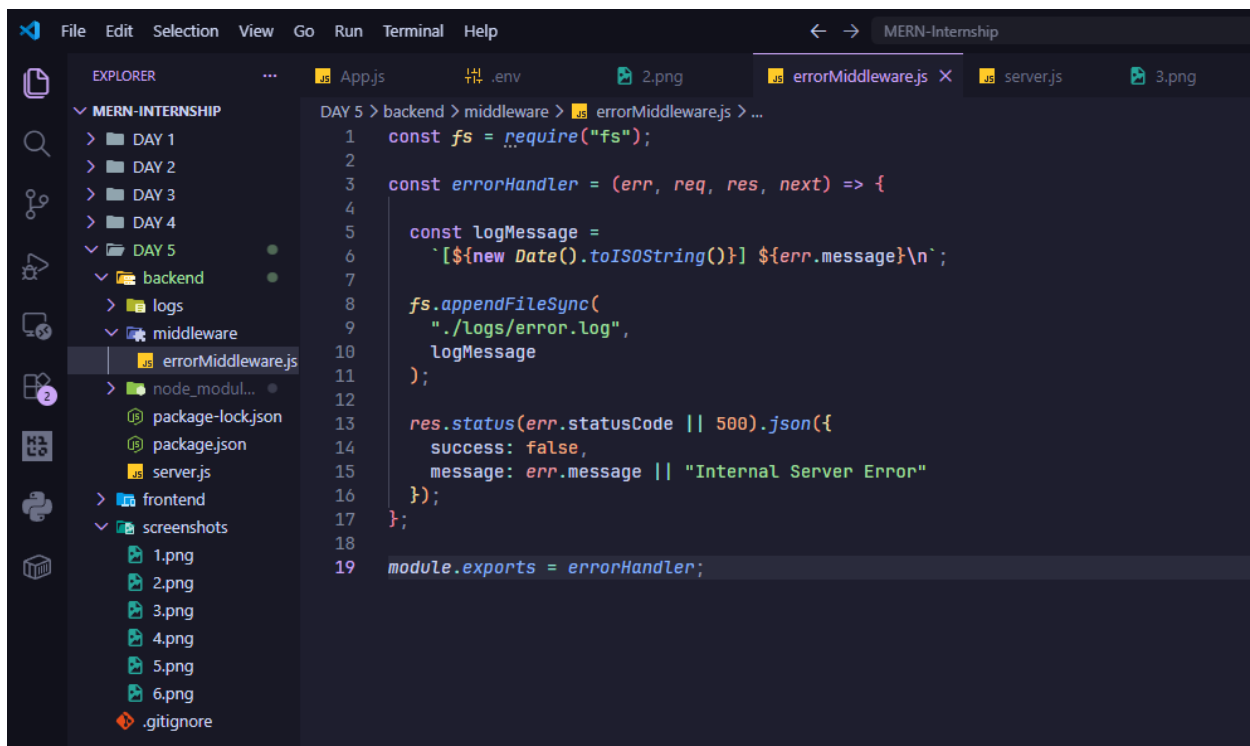
Implementation Steps

Step 1: Creating Error Middleware

A centralized middleware was created to:

- Catch application errors
- Generate structured JSON responses
- Log errors into a secure server log file

Screenshot 4: Error Middleware Code



```
DAY 5 > backend > middleware > errorMiddleware.js > ...
1  const fs = require("fs");
2
3  const errorHandler = (err, req, res, next) => {
4
5      const logMessage =
6          `[${new Date().toISOString()}] ${err.message}\n`;
7
8      fs.appendFileSync(
9          "./logs/error.log",
10         logMessage
11     );
12
13     res.status(err.statusCode || 500).json({
14         success: false,
15         message: err.message || "Internal Server Error"
16     });
17 }
18
19 module.exports = errorHandler;
```

Step 2: Creating API Routes

Two routes were implemented:

Success Route

/success

Returns a successful JSON response.

Error Route

/error

Generates a custom error and forwards it to the centralized middleware.

Step 3: Handling Invalid Routes

A route handler was added to catch undefined endpoints and generate a standard "Route Not Found" response.

API Testing Results

Success Route Test

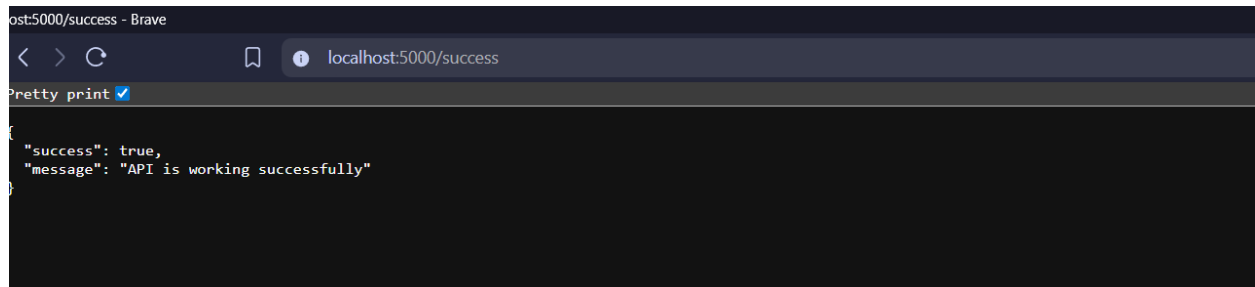
Endpoint:

<http://localhost:5000/success>

Response:

```
{
  "success": true,
  "message": "API is working successfully"
}
```

Screenshot 5: Success Route Response



Error Route Test

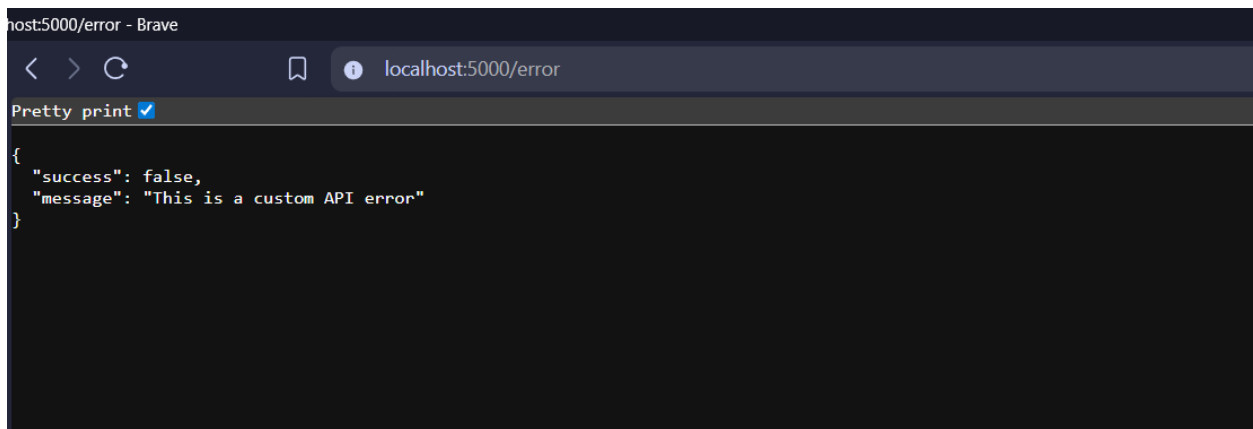
Endpoint:

<http://localhost:5000/error>

Response:

```
{
  "success": false,
  "message": "This is a custom API error"
}
```

Screenshot 6: Error Route Response



Invalid Route Test

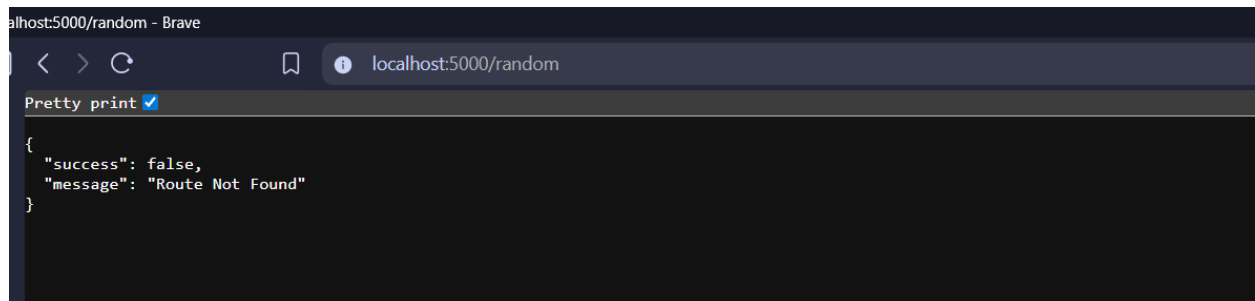
Endpoint:

<http://localhost:5000/random>

Response:

```
{
  "success": false,
  "message": "Route Not Found"
}
```

Screenshot 7: Invalid Route Response



Error Logging Verification

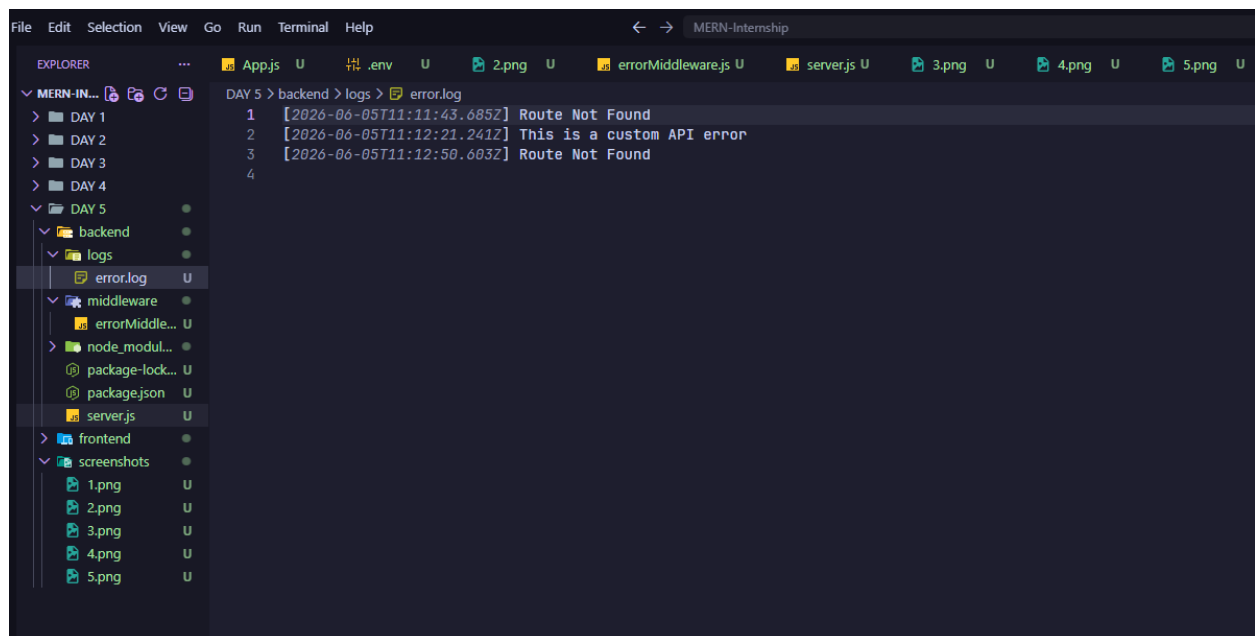
Errors generated during API execution were automatically stored inside:

`logs/error.log`

Example:

```
[2026-06-05T10:15:00.000Z] This is a custom API error
[2026-06-05T10:16:20.000Z] Route Not Found
```

Screenshot 8: Error Log File



Challenges Faced

- Understanding centralized middleware architecture.
- Managing structured error responses.
- Configuring environment variables correctly.
- Generating and verifying production builds.

Conclusion

Day 5 focused on preparing MERN applications for production environments and implementing professional backend error management practices. A React application was optimized using production build processes and environment variables, while the Express backend was enhanced with centralized error-handling middleware and secure error logging mechanisms. These practices improve application reliability, maintainability, and deployment readiness.

GitHub Repository

Repository Link: <https://github.com/chayan-soni/MERN-Internship>

Day 5 Folder:

```
DAY 5/  
├── frontend/  
├── backend/  
├── README.md  
└── .gitignore
```